

Task 1

1.1. SkyPulse Airshow & Aviation Expo

SkyPulse is a large public airshow containing flying demonstrations, drone demonstrations, exhibition areas, spectator areas, medical services, transport services and emergency- response teams.

The event is divided into several nested operational areas. Each area may contain smaller areas or individual operational units.

During the event, the control centre must react to situations such as:

- weather warnings;
- capacity limits;
- aircraft schedule changes;
- runway activation;
- transport delays;
- emergency evacuations;
- temporary pauses; and
- resumption of normal operations.

Different units respond differently to the same event notice.

For example, during a serious weather warning:

- an **AirDisplayStage** pauses flying;
- a **DroneDisplayUnit** lands its drones;
- an outdoor **VendorStall** suspends service;
- a **MedicalPost** remains operational;
- an **InformationScreen** displays the warning.

The design therefore uses:

Composite to represent the physical part-whole structure of the event.

Observer to distribute changing event information between operational areas and interested units.

This matches the practical's distinction that Composite answers what is contained in an event area, while Observer answers who needs to receive a change.

1.2. Composite Pattern

GoF role	SkyPulse Class
----------	----------------

Component	EventComponent
Composite	EventGroup
Leaf abstraction	EventUnit
Concrete Leaf	AirDisplayStage
Concrete Leaf	Gate
Concrete Leaf	VendorStall
Concrete Leaf	MedicalPost
Concrete Leaf	ShuttleStop
Concrete Leaf	InformationScreen
Concrete Leaf	DroneDisplayUnit
Concrete Leaf	FireResponseTeam
Client	Main.cpp

Observer Pattern

GoF Role	SkyPulse Class
Subject	Subject
Concrete Subject	EventControl
Concrete Subject	EventGroup
Observer	Observer
Concrete Observer	EventGroup
Concrete Observer	Interested EventUnit subclasses

Event Group participates in both patterns

As a Composite:

It owns and manages event components contained inside an area.

As an observer:

It can receive a notice from the area above it

As a subject:

It can pass that notice to interested observers below it.

1.4.a. The Composite is a genuine part whole tree because:

- SkyPulse contains Zones
- Zones contain Sectors
- Sectors contain operational units
- A sector can also contain another smaller group is required

b. Observer is required because the EventControl should not directly know every:

- Stage
- Vendor
- Medical post
- Gate
- Shuttle
- Drone unit

Instead, interested objects register themselves with appropriate subjects. When the event changed the Subject notifies the currently registered observers.

c. EventGroup owns its Composite children, if RunwaySector contains AirDisplayStage then RunwaySector is responsible for eventually destroying that AirDisplayStage.

A Subject does not own its Observer pointers. Registration means: send this object notifications.

d. Skypulse should use Push Observer, this is for when a change occurs the Subject send the relevant Notice Directly to the observer. This suits

TASK 2

2.4. Each EventGroup exclusively owns the EventComponent objects stored in its children collection. Ownership is therefore represented using composition. The EventGroup destructor deletes every child that remains in the collection. Since a child may itself be another EventGroup, destruction occurs recursively through the Composite tree. Consequently, deleting the root skyPulse object releases every owned group and leaf in the event tree. Each component has only one owning parent, ensuring that it is deleted exactly once.

When an object is transferred between groups, remove() removes the child from the old parent's collection without deleting it and returns the pointer. Ownership is temporarily relinquished and then transferred to the new parent using add(). The old parent can therefore no longer delete the transferred component, while the new parent becomes responsible for its eventual destruction. This prevents double deletion.

TASK 3

3.3. The SkyPulse EventFlow system implements several notice types to support ordinary operational changes, capacity-related changes, safety events and recovery situations. These notice types are OPEN_NOTICE, CLOSE_NOTICE, SCHEDULE_CHANGE, CAPACITY_ALERT, WEATHER_ALERT, TRANSPORT_DELAY, EVACUATE, RESUME and RUNWAY_ACTIVE. This exceeds the requirement of at least six notice types and includes all the required categories. Each notice is represented by a Notice object that stores a notice type, a descriptive message and a severity level. For example, a WEATHER_ALERT can contain the message "Strong winds exceed safe operating limits" with severity 2, while a RUNWAY_ACTIVE notice can contain the message "Emergency aircraft landing requested" with severity 3. The concrete Observer classes react differently to the same notice through their own implementations of update(const Notice&). For instance, during a weather alert an AirDisplayStage may pause its performance while a Gate may stop admitting attendees. During a runway emergency, the DroneDisplayUnit may land its drones, the InformationScreen may display the warning, and the FireResponseTeam may deploy. This allows the controller to distribute a common notice without needing to know how every individual unit will respond.

3.5. The SkyPulse EventFlow system uses the **Push Observer approach**. In this approach, the Subject sends the complete Notice object directly to every registered Observer through the `update(const Notice& notice)` operation. The information pushed to the Observer therefore consists of the notice type, message and severity. The specification allows either push or pull, but requires the chosen approach to be used consistently and the transferred state to be clearly explained. In our implementation, when EventController issues a notice, it calls `notify(notice)`, which in turn calls `update(notice)` on each registered Observer. When an EventGroup receives the notice, it also acts as a Subject and forwards the same notice to its own registered observers. This allows the notification to cascade from EventController to SkyPulse, then to Airside, then to lower-level groups such as Runway, and finally to concrete units such as AirDisplayStage and Gate.

The Push approach was chosen because the Notice object is small and already contains the information required by the concrete event units to make a decision. An Observer can simply inspect the notice type, message and severity without querying the Subject for additional state. This reduces coupling because concrete units do not need to know which specific Subject sent the notice. A Pull approach would require `update()` to inform an Observer that something changed and then require the Observer to query the Subject, for example by calling a method such as `getCurrentNotice()`. Although Pull can be useful where a Subject contains large amounts of state and different Observers require different pieces of that state, it would make the SkyPulse implementation more complex. The disadvantage of Push is that every Observer receives the entire Notice even if it only requires part of it, but this is acceptable in our design because the Notice contains only a small amount of information. The Push approach is therefore simpler and more suitable for the event notification requirements of SkyPulse, and it is also shown in the sequence diagrams through calls such as `update(weather)` and `update(emergency)`.

Task 4

4.4. Three original features

Severity-aware weather reaction

Notice carries a severity value. Only severity ≥ 3 pauses the air display, only severity ≥ 4 suspends outdoor vendor service. The threshold logic sits inside each leaf's `update()`, not in a central switch.

Cascading notification through Composite levels

EventGroup is both Observer and Subject. On `update()` it applies its own reaction and then `notify()`s registered children, so a single Control notice reaches Zone then Sector then Leaf without hard-coded loops in main.

Interactive tree printer and runtime attach/detach

`main.cpp` prints the live Composite tree, issues any of nine notice types, attaches/detaches observers, and transfers units at runtime - proving registration and ownership stay consistent after structural change.

None of these features turn a single class into a god object: severity belongs to Notice, cascade to EventGroup, and interaction to the thin main client.